Robotics and Computer-Integrated Manufacturing 27 (2011) 994–1000

Contents lists available at ScienceDirect

Robotics and Computer-Integrated Manufacturing

journal homepage: www.elsevier.com/locate/rcim

A virtual RV-M1 robot system

Rajesh Kumar [a,*], Parveen Kalra [b], Neelam R. Prakash [c]

[a] Mechanical Engineering Department, Room No. 403, Academic Block 2, U.I.E.T., Panjab University, Chandigarh 160014, India
[b] Production Engineering Department, PEC University of Technology, Chandigarh, India
[c] Department of Electronics & Electrical Communication, PEC University of Technology, Chandigarh, India

ARTICLE INFO

Article history:
Received 19 May 2009
Received in revised form
16 April 2011
Accepted 12 May 2011
Available online 17 June 2011

Keywords:
RV-M1 robot
MATLAB
Simulation
Kinematics
Teachbox

ABSTRACT

Exploring a virtual model under simulated environments is the best way to learn about a real system. This is particularly true in robotics where it is quite expensive to provide the system to each individual. The interdisciplinary area of robotics is being studied commonly in various fields like electrical, computer, mechanical engineering, nanotechnology, etc. A virtual robot system can help one fully understand the controls and working of a robot. The system may also be helpful to design the path and plan the trajectory of a robot in an industrial environment or other robotics application. Virtual model of RV-M1 robot has been developed in the MATLAB environment. The virtual system performs forward kinematics and inverse kinematics in addition to providing a simulation of the robot teachbox.

© 2011 Elsevier Ltd. All rights reserved.

1. Introduction

Education and learning using a virtual system is a good alternative to actual hands-on-experience. Simulation is a process of designing a model of a real system and performing experiments with this model to understand the behavior of the system. Robotics is a field where a robot simulation program may be used for teaching robotics in the classroom or in environments that are not conducive to purchasing and maintaining an industrial robot. There is a strong presence of the MATLAB in the academic community and a virtual robot system in MATLAB software, which exploits the strong functionality of the software at no extra cost is the need of the hour. The details about the work done in the area of robot simulation can be found in [1]. The robot model developed in [2,3] displays the robot links as lines, which result in poor visualization of kinematics simulation. Recently Pro-E modeling has also been used but this requires the availability of the Pro-E Software.

The academic value of the RV-M1 robot is still very high in spite of its limited functionality and software control. In view of the replacement cost involved, many institutions still use the Mitsubishi RV-M1 robot in their laboratories. Training on virtual system would be a valuable learning tool in these academic institutions. The objective of this work was to develop a simulation software for Mitsubishi RV-M1 robot, which would represent the robot as a three-dimensional model. The robot model and the simulations have been developed using the MATLAB programming. The language chosen for this task is MATLAB due to its special features. The advantage of using MATLAB is well known and widely recognized due to the comprehensive library of standard and powerful graphics MATLAB functions [4–9]. MATLAB has been extensively used in building the virtual reality world of various simulations. More information about MATLAB may be found in "*Matlab User Guide*" [http://www.mathworks.com]. The organization of this paper is as follows: Section 2 describes the kinematics structure of the RV-M1 robot. A closed form solution for the robot is developed in this section. Section 3 is focused on robot modeling and development of the program. Results have been discussed in Section 4 while conclusion has been made in Section 5.

2. Kinematics of RV-M1 robot

A robot is a series of links and joints, which creates a kinematic chain. Each link connects two adjacent joints, and each joint connects two adjacent links. A coordinate frame needs to be established for every link of the robot. Once the task is accomplished, a set of transformations may be determined that will take us from one joint frame to the next. If all these transformations are combined from frame 0 to frame n, the entire robot transformation matrix ${}^0A_n$ is defined. A local reference frame for every link has been given by assigning a Z-axis and an X-axis. A revolute joint will have a rotation about Z-axes whereas a prismatic joint will have translation along Z-axis. X-axis is taken along the common normal to the two consecutive Z-axes. The Y-axis is simply the cross product of the other two axes.

The kinematics modeling i.e. forward and inverse kinematics solutions for the Mitsubishi RV-M1 robot is now developed. To define links locations, coordinate systems or frames are

* Corresponding author. Mobile: +09780826926.
E-mail address: rajeshmadan@rediffmail.com (R. Kumar).

0736-5845/$ - see front matter © 2011 Elsevier Ltd. All rights reserved.
doi:10.1016/j.rcim.2011.05.003

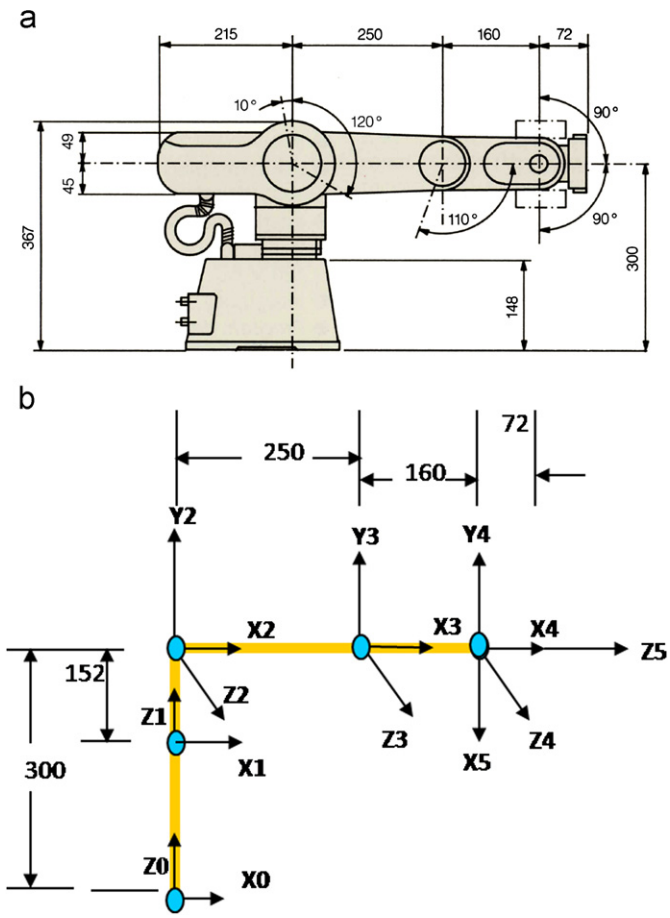

Fig. 1. Mitsubishi RV-M1 robot and its coordinate frames established (all dimensions are in mm).

Table 1
DH parameters for RV-M1 robot.

Links	a_n (mm)	α_n (rad)	θ_n (rad)	d_n (mm)
1	0	pi/2	θ_1	152
2	250	0	θ_2	0
3	160	0	θ_3	0
4	72	pi/2	θ_4	0
5	107	θ_5	0	0

assigned to each link [10]. The link coordinates frames of the Mitsubishi RV-M1 robot are shown in the Fig. 1.

Using the DH convention, the DH parameters a_n, α_n, θ_n and d_n of the robot link can be determined [11,12]. For the Mitsubishi RV-M1 robot, having all revolute joints, the parameters a_n, α_n, and d_n are fixed and only θ_n is variable. The DH parameters have been established for RV-M1 robot and are shown in Table 1 [13,14]

The arm matrix for each link may be determined by the equation

$$^{n-1}T_n = A_n = Rot(Z,\theta_n)Trans(Z,d_n)Trans(X,a_n)Rot(X,a_n) = \begin{bmatrix} C\theta_n & -S\theta_n C\alpha_n & S\theta_n S\alpha_n & a_n C\theta_n \\ S\theta_n & C\theta_n C\alpha_n & -C\theta_n S\alpha_n & a_n S\theta_n \\ 0 & S\alpha_n & C\alpha_n & d_n \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

Here, $C(\theta)$ denotes $\cos(\theta)$ and $S(\theta)$ denotes $\sin(\theta)$.

The arm matrix once known permits the following transformations:

$$P_{n-1} = {}^{n-1}A_n P_n \quad (2)$$

Using Eq. (2), the homogeneous coordinates of point P in frame n can be transformed to frame $n-1$. The representation of nth frame with respect to base frame is obtained by post-multiplication of arm matrices and is given by

$$^{0}T_n = {}^{0}A_1^1 A_2^2 A_3 \ldots {}^{n-1}A_n \quad (3)$$

The inverse kinematics problem for the Mitsubishi RV-M1 robot is solved in two parts.

The angle of rotation of joint 1 (θ_1: rotation about base of the robot) is the angle through which the base motor must be moved to align the arm along the general direction of the desired point.

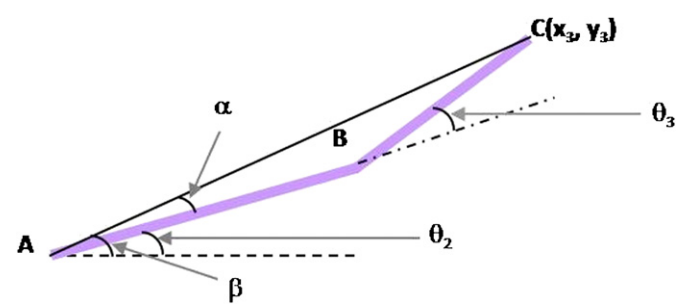

Fig. 3. Inverse kinematics solution for robot RV-M1 (Here, $\alpha+\theta 2=\beta$).

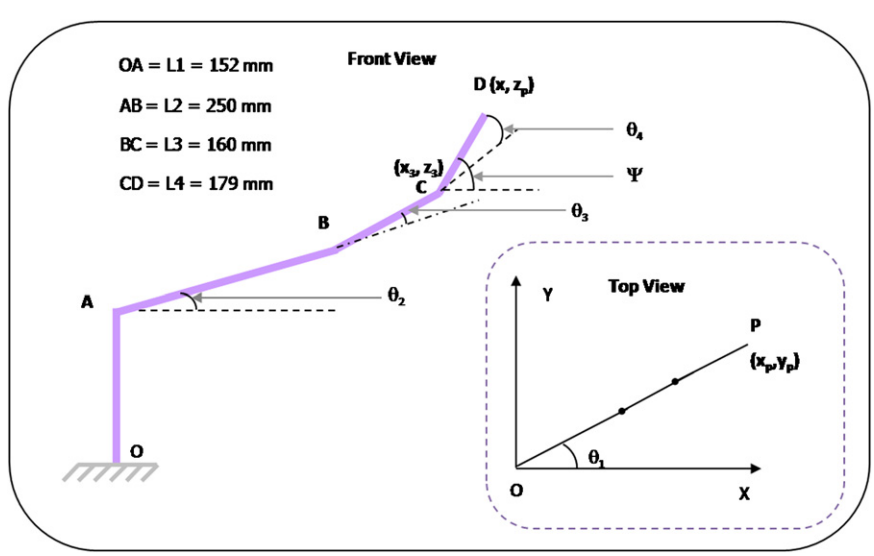

Fig. 2. Inverse kinematics solution for robot RV-M1 (top view and front view).

Point P in Fig. 2 represents the robot gripper tip with coordinates x_p, y_p and z_p. The angle θ_1 is calculated as

$$q_1 = \arctan(y_p/x_p) \tag{4}$$

The upper arm, forearm and the gripper lie in a plane, the angles θ_2, θ_3 and θ_4 can be determined by considering the robot as 3 R planar manipulator as shown in Fig. 2. Here, point O is the origin, point A is the shoulder joint, point B is the elbow, point C is the wrist and point D is the gripper tip. The relationship between x, x_p, y_p is

$$x = [(x_p)^2 + (y_p)^2]^{1/2}. \tag{5}$$

Using Fig. 3, coordinates of the tip of the robot gripper, P are

$$x = L_2\cos(\theta_2) + L_3\cos(\theta_2+\theta_3) + L_4\cos(\theta_2+\theta_3+\theta_4) \tag{6}$$

$$z_p = L_2\sin(\theta_2) + L_3\sin(\theta_2+\theta_3) + L_4\sin(\theta_2+\theta_3+\theta_4) \tag{7}$$

When defining the position of the end of the arm, x, z_p and Ψ will be used. The angle Ψ is the orientation angle for the wrist. The position of C ($x3$, $z3$) can be obtained by the following equations:

$$x_3 = x - L_4\cos(\Psi) \tag{8}$$

$$z_3 = z_p - L_4\sin(\Psi) \tag{9}$$

Having determined the position of the joint 4, the angles θ_2 and θ_3 are determined using the methodology of a two degree of freedom manipulator.

The world space coordinates for the end of the arm (x_3, z_3) would be

$$x_3 = L_2\cos(\theta_2) + L_3\cos(\theta_2+\theta_3) \tag{10}$$

$$z_3 = L_2\sin(\theta_2) + L_3\sin(\theta_2+\theta_3) \tag{11}$$

Using the trigonometric identities $[\cos(A+B) = \cos(A)\cos(B) - \sin(A)\sin(B)]$ and $[\sin(A+B) = \sin(A)\cos(B) + \cos(A)\sin(B)]$, the equations (10) and (11) now may be written as

$$x_3 = L_2\cos(\theta_2) + L_3\cos(\theta_2)\cos(\theta_3) - L_3\sin(\theta_2)\sin(\theta_3) \tag{12}$$

$$z_3 = L_2\sin(\theta_2) + L_3\sin(\theta_2)\cos(\theta_3) - L_3\cos(\theta_2)\sin(\theta_3) \tag{13}$$

By squaring both sides and adding, θ_3 can be calculated as

$$\cos(\theta_3) = [(x_3)^2 + (z_3)^2 - (L_2)^2 - (L_3)^2]/[2(L_2)(L_3)] \tag{14}$$

Defining, $\theta_2 = \beta - \alpha$, as shown in Fig. 3

$$\tan\alpha = [(L_3)\sin(\theta_3)]/[(L_3)\cos(\theta_3) + (L_2)] \tag{15}$$

$$\text{and } \tan\beta = z_3/x_3. \tag{16}$$

Since $\Psi = (\theta_2+\theta_3+\theta_4)$, θ_4 can be calculated as

$$\theta_4 = \Psi - (\theta_2+\theta_3). \tag{17}$$

3. Robot modeling and development of program

3.1. Robot modeling

The robot model has been developed by making individual links in the local frame and transforming it to base frame using DH matrices. The links are synthesized using a modified version of the MATLAB cylinder function, surf and fill3 functions. The links are defined in the link frame of the robot. To completely describe the transformation from one frame to another, the coordinate frames for every link of the robot have already been established. The DH parameters for every link have been defined. Using composite transformation, the program calculates the final position of each link with respect to the robot reference frame.

3.2. Virtual teachbox

Virtual teachbox has been developed using MATLAB Graphical User Interface (GUI). First, main GUI body file has been made where GUI elements like pushbutton, toggle button, editable text, etc. have been defined along with corresponding callbacks. The callback files have been developed to perform the desired action for the GUI elements defined in the GUI body file.

3.3. Callbacks associated with teachbox

The callback files have been developed to perform the desired action for the GUI elements defined in the GUI body file. The desired action has been defined in MATLAB language. For example by pressing the [< O > /0] button of the virtual teachbox when robot teachbox is in on state, the associated callback is activated and virtual robot opens its gripper.

3.4. Pick and place application interface

Additional pick and place application has been incorporated. The GUI elements for this application are defined in main GUI body file itself and appear in lower left corner of the virtual robot window(below the virtual teachbox). By clicking these buttons, desired action is performed as coded in corresponding associated

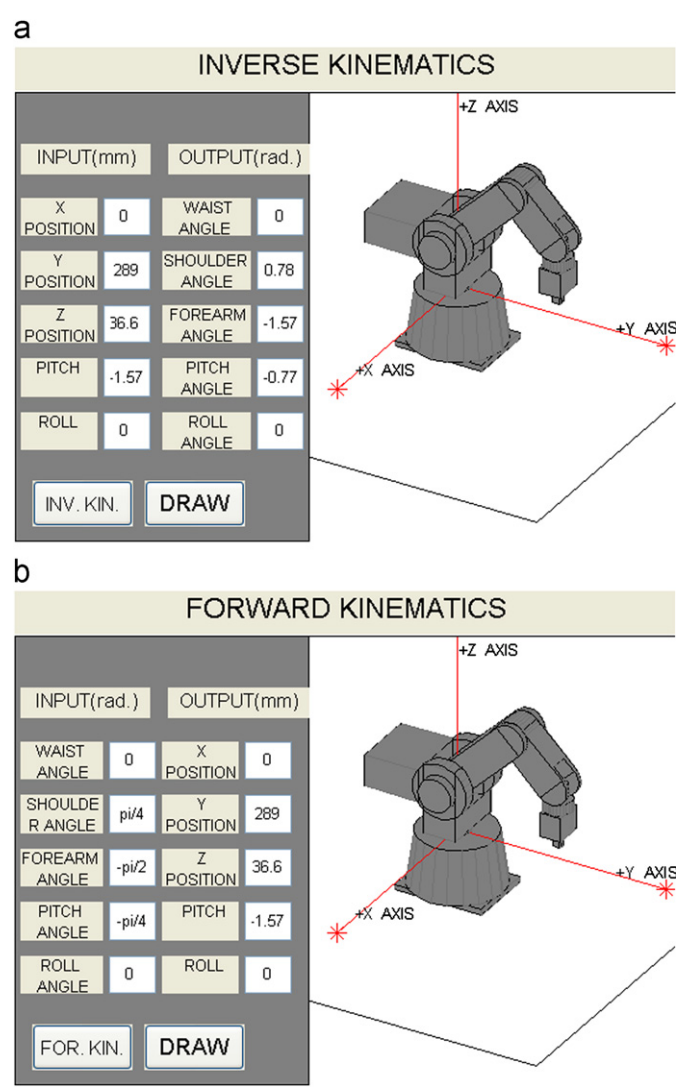

Fig. 4. Functioning of forward and inverse kinematics.

callbacks. The files for modeling and manipulating three-dimensional objects using robot gripper have also been made for pick and place applications.

3.5. Execution of software

Forward and inverse kinematics simulation of the robot can be invoked by entering 'forwardkin' and 'inversekin' at the MATLAB command prompt. In the forward kinematics user interface, the input parameters have to be given by the user in the form of joint angles and the software calculates the griper's position and orientation. In the inverse kinematics user interface, the input parameters have to be given by the user as gripper's position and orientation and the software calculates the desired parameters in the form of joint angles. On pressing *DRAW* Button, the 3-d robot will also appear in the graphics window. It is assumed that the wrist roll angle would be supplied by the user.

The virtual teachbox simulation can be invoked by entering 'robogui' at the MATLAB command prompt. The virtual teachbox appears on the left side of the figure while the 3-d robot will appear on right side under robot draw area. Buttons for pick and place application are provided in the lower left corner. The left bottom place is provided for specifying the object's dimension, present location and targeted location.

Now directly with the help of mouse click, the virtual teachbox can be accessed. The three-dimensional model of robot will appear in robot draw area by pressing *ON/OFF* button. The working is discussed in details in the next section.

4. Simulation results

In the forward kinematics dialog box, putting waist angle=0, shoulder angle=pi/4, forearm angle=−pi/2, pitch angle=−pi/4 and pressing the draw button, the software calculates the desired position of the hand using the forward kinematics and draws the robot in that position. The calculated location and orientation of the robot's gripper will be displayed under the field output parameters in the forward kinematics window as shown in Fig. 4.

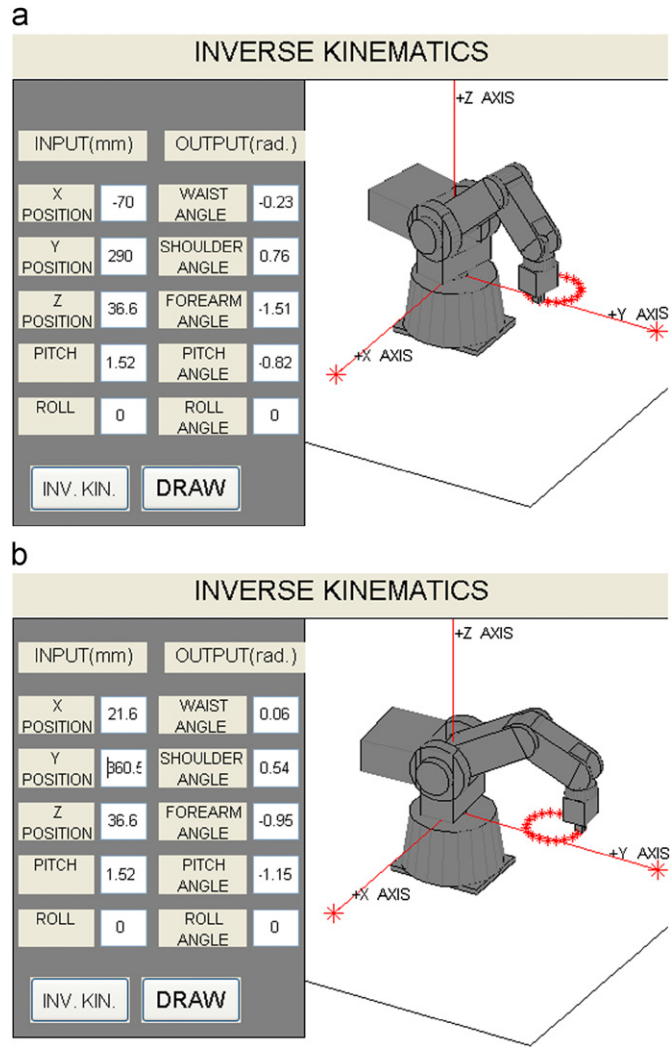

Fig. 6. Virtual robot showing circular trajectory.

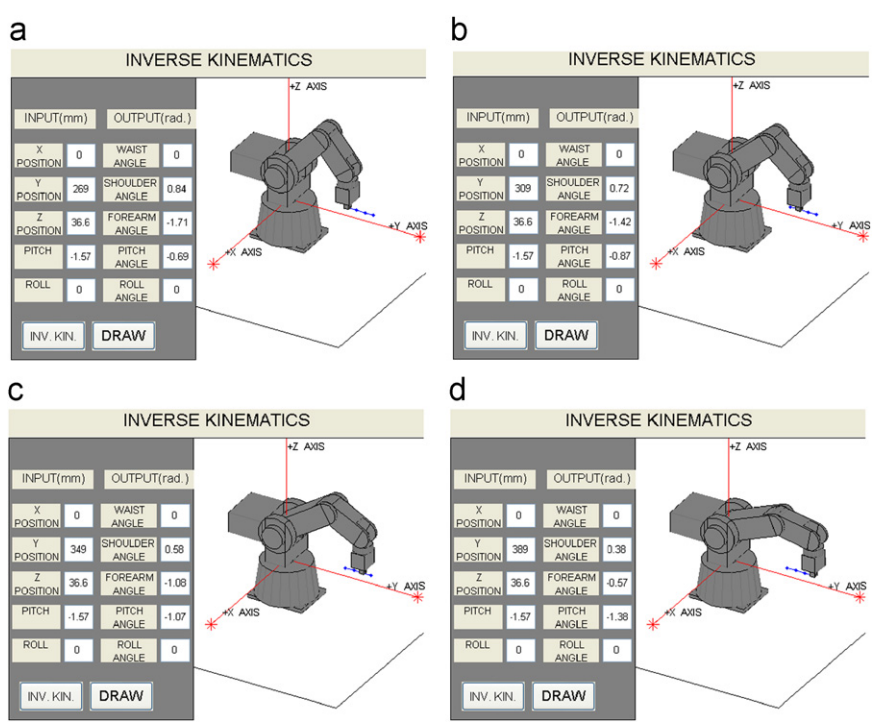

Fig. 5. (a–d) Virtual robot showing straight line trajectory.

In the inverse kinematics user interfaces, the input parameters have to be given by the user as the gripper's position and orientation and the software calculates the desired parameters as joint angles. Putting $x=0$, $y=289$, $z=36.6$, pitch$=-$pi/2, roll$=0$ in the inverse kinematics dialog box pressing the draw button, the software calculates the desired position of the hand using inverse kinematics and draws the robot in that position as shown in inverse kinematics windows of Fig. 4.

Further, the inverse kinematics calculations were carried out for implementing a straight line trajectory of the robot gripper along the line with end points (0, 249 and 36.6) mm and (0, 389 36.6) mm. The robot configurations at initial, final and intermediate points are shown in Fig. 5.

Moreover the inverse kinematics calculations were carried out for implementing other path trajectories like one shown in Fig. 6 demonstrating circular path trajectory.

The virtual teachbox can be invoked by writing ***robogui*** at MATLAB command prompt and then pressing enter button. When ***ON/OFF*** button of the virtual teachbox is pressed with a mouse click, the 3-d robot appears in virtual robot window as shown in the Fig. 7.

When *EMERGENCY* button on the Teachbox is pressed, the message "error mode 1" is shown in the display bar. Point-to-point Mode instructs the virtual robot to jog (move) using its individual joints (axes) when the appropriate jog buttons are pressed. To select *PTP* mode, the *PTP* button on the robot teachbox is activated. All subsequent commands will be performed in *PTP* Mode. For example, if the $Y+/S+$ button is pressed when the robot is set in *PTP* mode, the button will be recognized as $S+$. The upper arm section will move in upward direction, pivoting around the shoulder joint. By pressing the PTP button once again, *PTP* mode is canceled.

XYZ mode instructs the virtual robot to jog (move), using the gripper's tip as a reference point, for moving around a three-dimensional X, Y and Z grid system. To select ***XYZ*** mode, the ***XYZ*** button on the robot teachbox is activated. For example, if the $\boldsymbol{Y+/S+}$ button is pressed when the robot is set in ***XYZ*** mode, the button will be recognized as $Y+$. The gripper's tip will move along the positive Y direction. All the sections and joints of the robot move independently of each other to keep the gripper's tip as close to this Y-axis as possible. All subsequent commands will be performed in ***XYZ*** mode. To cancel ***XYZ*** mode, click on the button once more.

On invoking the virtual teachbox, the PTP state is the default state. Hence any movement command instructs the virtual robot to jog (move) using its individual joints (axes) when the appropriate jog buttons are pressed. By pressing a series of jog buttons, the robot takes the position as shown in the Fig. 8.

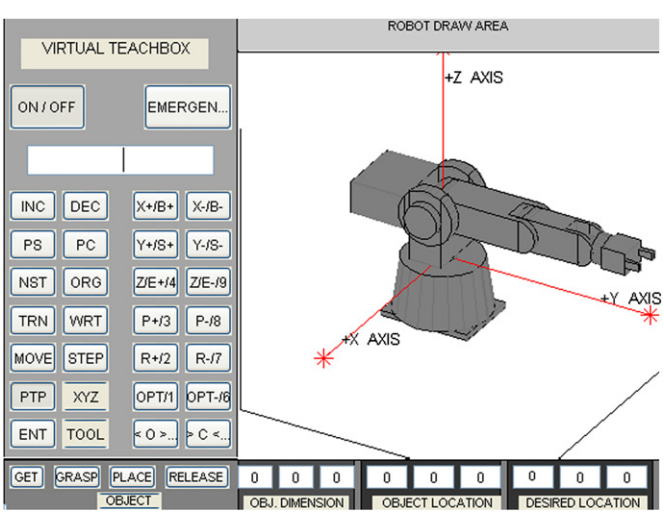

Fig. 7. Virtual robot with teachbox.

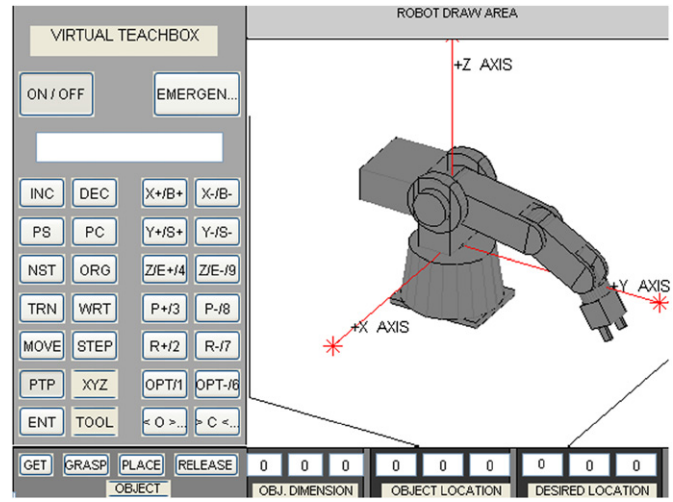

Fig. 8. Jog mode.

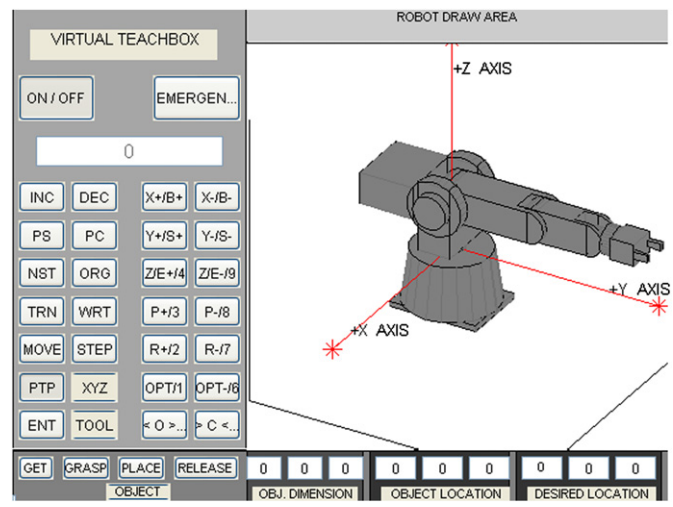

Fig. 9. ORG followed by ENT.

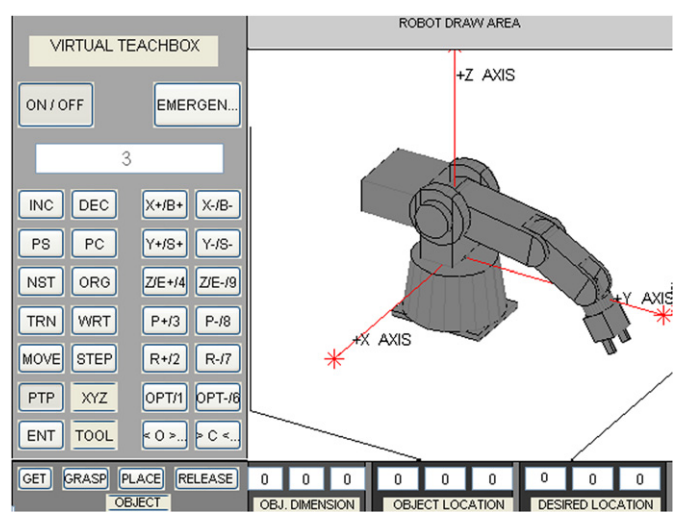

Fig. 10. *MOVE* followed by *ENT*.

Pressing *PS* button and *3* number button in virtual teachbox followed by *ENT button* stores this position in memory at number *3*. To bring the robot, back to its original outstretched position, the

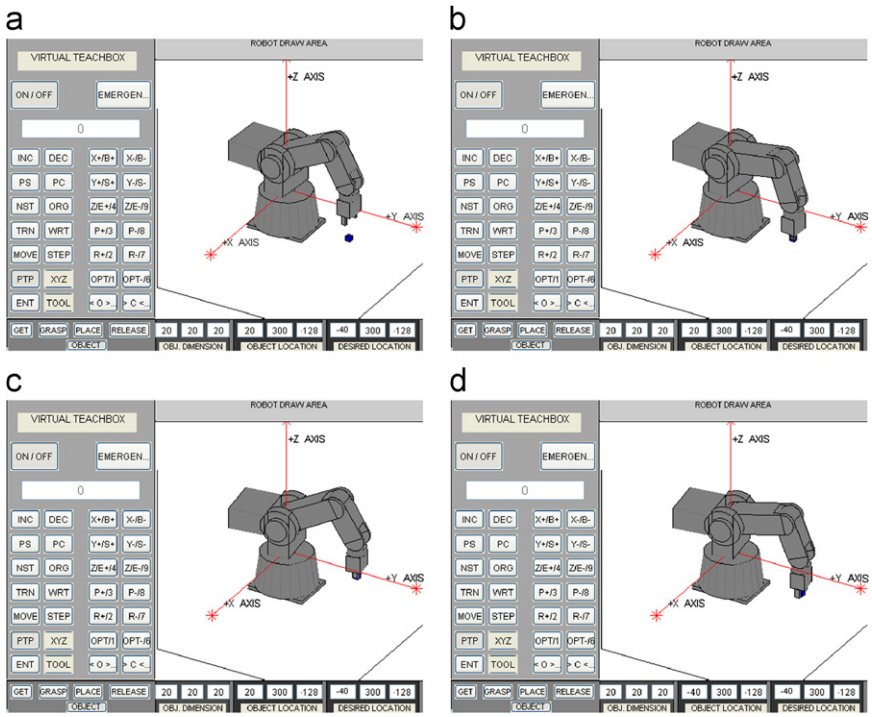

Fig. 11. (a–d) Virtual robot showing pick and place application using four additional buttons *GET, GRASP, PLACE and RELEASE*.

ORG button is pressed followed by *ENT button. The robot configuration corresponding to the ORG state is displayed as shown in* Fig. 9. To move the gripper to previous stored memory location say 3, *Move* button and *3* number button may be pressed followed by *ENT* as shown in Fig. 10.

The robot can grip the object situated anywhere in the robot's workspace and place it at the desired location within workspace given by the user with the help of four additional buttons *GET*, *GRASP*, *PLACE* and *RELEASE*. These buttons (which do not exist in actual robot teachbox) have been developed for the simulation of pick and place application. The dimension of the object with its initial location can be entered at the specified place in the Graphical User Interface window.

When *GET* button is pressed, the software calculates the position of the gripper and moves the virtual robot's gripper vertically above the desired position as shown in the Fig. 11a. By pressing *GRASP* button, the object is gripped in the robot hand as shown in the Fig. 11b. When *PLACE* button is pressed, the software calculates the position of the set-down point above the final location specified by the user and positions the object at that location. When *RELEASE* button is pressed, the robot hand releases the object at the desired position by opening its gripper as shown in Fig. 11c and d.

5. Conclusions and future scope

Commercial simulation softwares for robot system are very expensive. Therefore, a kinematics simulation of Mitsubishi RV-M1 robot using MATLAB can be very useful. This software enables a user to learn the kinematics of a Mitsubishi robot by displaying the three-dimensional robot model. The three-dimensional robot gives a good visualization of the robot arm.

The development of simulation software for Mitsubishi RV-M1 robot in MATLAB was a novel approach in this direction. There is a wide range of possibilities for extensions of the system and several interesting openings for future research and development exist. Software may be developed for the dynamic simulation of the Mitsubishi RV-M1 robot. A Simulink MATLAB toolbox could be used for the dynamic simulation of the robot at present. Software may be developed to perform a robot controller inbuilt functions like PT (pallet), WH (Where), MP (Move position), DW (Draw), MC (Move continuous), SP (Speed), etc. Software may be developed to simulate a robot workcell. This would include features like trajectory planning, modeling of other components of the workcell, etc. An interface could be developed to make it possible to have a common system i.e. MATLAB software for both simulation of the virtual robot and execution of actual robot. Though the programming in the MATLAB environment would be a useful tool in view of the strong presence of MATLAB in academic community, programming using RV-M1 language could be used as an immediate tool for actual robot operation.

References

[1] Jochheim A, Gerke M, Bischo A. Modeling and simulation of kinematic systems. Control Systems Engineering group, Department of Electrical Engineering, University of Hagen; Aug. 27, 1999.
[2] Ferrira NMF, Machado JA Machado T. ROBLIB: an educational program for robotics. In: Proceedings of the symposium on robot control vol. 1. Vienna, Austria; Sept 2000. p. 163–68.
[3] Corke PI. A computer tool for simulation and analysis: the robotic toolbox for matlab. In: Proceedings of the national conference of the Australian robot association, Melbourne; July 1995. p. 319–30.
[4] Matlab User Guide. The Mathworks Inc.,USA, 〈http://www.mathworks.com〉.
[5] Crane ARC, Crane CM. Review of Matlab Version 5.1. De Montfort University.
[6] Demoyer R, Mitchell EE. Use of the Matlab graphical user interface development environment for some control system applications. In: Proceedings of the 29th ASEE/IEEE frontiers in education conference; Nov. 1999. p. 12b3-7–11.
[7] Farrell J. MATLAB as an Engineer's Problem Solving Tool. Riverside: College of Engineering, University of California; January 7, 2003.
[8] Stephen J. MATLAB Programming for Engineers. Brooks/Cole; 2000.

[9] Trivailo P. Matlab in Building Virtual Reality Worlds: Animated Dynamic of Elastic Space Structures and Advance Robotic System, Department of Aerospace Engineering, RMIT University, Melbourne, Australia.
[10] Wernholt E. Rigid Motions and Forward Kinematics, October 2, 2002.
[11] Fu KS, Gonzalez RC, Lee CSG. Robotics: Control, Sensing, Vision, and Intelligence. New York: McGraw-Hill; 1987.
[12] Schilling R. Fundamentals of Robotics. Upper Saddle River, NJ: Prentice Hall; 1990.
[13] Kumar R, Kalra P, Prakash NR. Kinematics Simulation of Mitsubishi RV-M1 Robot. In: Proceedings of the national conference on advances in manufacturing technology. Nov. 9–11, 2005.
[14] Kumar R, Kalra P, Prakash NR. Virtual teachbox for Mitsubishi RV-M1 robot. In: Proceedings of the IEEE international conference on advanced computing. March 6–7, 2009.